

Aufgabe 1 (3)

Gegeben sei eine Methode main in einer Java Klasse.

```
public static void main(String[] args) {  
    /* body */  
}
```

Die folgenden Anweisungen sollen als “Body” (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden. Geben Sie für jede Anweisung an, was für eine Ausgabe erzeugt wird – entweder was gedruckt wird, oder ob ein Laufzeitfehler auftritt (schreiben Sie “Exception”), oder ob der Compiler einen Fehler feststellt (schreiben Sie “Compile-Fehler”). Achten Sie auf die korrekte Formatierung der verschiedenen Typen, also z.B. 7.0 statt 7 für eine reelle Zahl (double).

1. `System.out.println(11 + 16 / 4 * 2 + (4 + ">") + 4 * 2);`

2. `System.out.println(20 / 10 % 6 + 3 / 2 + (double) 5 / 4 + 3 / 4);`

3. `System.out.println( ( 11 % 4 ) > 2 && 9 > (16 / (2 / 4)) && 1 % 8 < 0);`

Aufgabe 2 (6)

Gegeben sei eine Java Klasse mit einer Methode `foo(...)`, die mit verschiedenen Argumenten aufgerufen wird.

```
import java.util.Arrays;

class J02_s22 {
    public static void main(String[] args) {

        int [][] a = {{1, 2, 3}, {3, 2, 1}, {4, 5, 6}};
        System.out.println(Arrays.toString(a[0])); // [ 1, 2, 3 ]

        int i = 0;
        foo(a, i);

        System.out.println(Arrays.toString(a[0])); //

        System.out.println(Arrays.toString(a[1])); //

        i = i + 1;
        foo(a, i);

        System.out.println(Arrays.toString(a[0])); //

        System.out.println(Arrays.toString(a[1])); //

        int [][] b = new int[3][0];
        int [] c = {4, 5, 6};
        b[0] = c;

        i = 0;
        foo(b, i);

        System.out.println(Arrays.toString(b[0])); //

        System.out.println(Arrays.toString(b[1])); //
    }

    static void foo(int[][] x, int y) {
        x[y][y+1] = x[y][y];
        y = y + 1;
    }
}
```

Bitte geben Sie rechts neben den `println()` Statements an, was diese ausgeben. Bitte geben Sie auch ggf. ausgegebene Klammern an. Um das Format in Erinnerung zu rufen haben wir die Ausgabe des ersten `println()` Statements bereits angegeben. Sollten Anweisungen nicht ausgeführt werden können, so markieren Sie bitte diese Anweisungen deutlich und schreiben rechts "Laufzeitfehler" (oder "Exception"); Sie brauchen den genauen Fehler bzw. die genaue Exception nicht angeben.

>

Aufgabe 3 (4)

Gegeben sind die Precondition und Postcondition für das folgende Programm

```
public int compute(int v, int n) {  
    // Precondition:  $n \geq 0$   
  
    int x;  
    int tmp;  
  
    x = 1;  
    tmp = 1;  
  
    // Loop Invariante:  
  
    while (x <= n) {  
        tmp = tmp * v;  
        x = x + 1;  
    }  
  
    // Postcondition:  $\text{tmp} == 2^n$   
  
    return tmp;  
}
```

Bitte geben Sie die Loop Invariante an.

Loop Invariante: _____

Aufgabe 4 (7)

Vervollständigen Sie die Lücken im untenstehenden Programmcode, sodass die main Methode in der InheritanceTest Klasse ohne Fehler kompiliert und ausgeführt werden kann, ohne eine Exception zu werfen. Der zu erwartende Konsolen-Output von main ist unten angegeben. Sie dürfen keine weiteren Klassen, Methoden, oder Interfaces hinzufügen.

Tipp: Nicht in allen Lücken *muss* etwas stehen aber in allen Lücken *kann* etwas stehen.

```
class InheritanceTest {  
  
    public static void main(String[] args) {  
        Z ref1 = new B();  
        ref1.bar();  
        System.out.println("++");  
        Z ref2 = new A();  
        ((A) ref2).bar();  
        System.out.println("++");  
  
        C c1 = new C();  
        System.out.println("C.foo():");  
        c1.foo();  
        System.out.println("--");  
        D d1 = new D();  
        if (d1 instanceof C) {  
            ((C)d1).test();  
        } else {  
            d1.foo();  
        }  
    }  
}
```

Jedesmal wenn die Methode main in der Klasse InheritanceTest ausgeführt wird (alle Klassen sind in dem selben Package), sollte folgender Output auf die Konsole geschrieben werden:

```
Hello  
Bingo  
++  
Hello  
++  
C.foo():  
Here  
--  
Found
```

```

class C ----- {

    public void foo() {
        System.out.println("Here");
    }

    public void test(){
        System.out.println("Test");
    }

}

class D ----- {

    public void foo() {
        super.foo();
    }

}

class Z ----- {
    public void bar() {
        System.out.println("Hello");
    }
}

class A ----- {

    int a1 = 0;

    A() {}
    A(int v) {
        a1 = v;
    }

    public void foo() {
        System.out.println("Found");
    }

}

class B ----- {

    B() { }

    B(int w) {
        super(w);
    }

    public void bar() {
        super.bar();
        System.out.println("Bingo");
    }

}

```

Aufgabe 5 (9)

Gegeben seien diese Klassen und Interfaces in separaten Dateien (im default Package):

```
class Caniformia {
    String name = "Hundeartige";
    String shortName = "HA";

    public String toString() {
        return shortName;
    }
}

class Canidae extends Caniformia {
    String name = "Hunde";
    String shortName = "H";

    public void jagdbar() {
        System.out.println("J2_" + shortName);
    }

    public void geschuetzt() {
        System.out.println("G2_" + shortName);
    }
}

class Arctoidea extends Caniformia {
    String name = "NichtHunde";

    public void jagdbar() {
        System.out.println("J3_" + shortName);
        praesent();
    }

    public void praesent() {
        System.out.println("P3_" + shortName);
    }
}

class Ursidae extends Arctoidea {
    String name = "Baeren";
    String shortName = "B";

    public void geschuetzt() {
        System.out.println("G4_" + shortName);
    }
}

class Pinnipedia extends Arctoidea {
    String name = "Robben";
    String shortName = "R";

    public void geschuetzt() {
        System.out.println("G5_" + shortName);
    }

    public void praesent() {
        System.out.println("P5_" + shortName);
    }

    public String toString() {
        return shortName;
    }
}

class Otariidae extends Pinnipedia {
    String name = "Ohrenrobbe";
    String shortName = "O";

    public void geschuetzt() {
        super.geschuetzt();
        System.out.println("G6_" + shortName);
    }
}

class Odobenidae extends Pinnipedia {
    String name = "Walrosse";
    String shortName = "W";

    public void geschuetzt() {
        System.out.println("G7_" + shortName);
    }
}
```

In einer Klasse Explore in dem selben Package befindet sich die Methode main.

```
public static void main (String[] args) {

    /* Body */

}
```

Die folgenden Anweisungen sollen als "Body" (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden. Geben Sie für jede Anweisungsfolge an, was für eine Ausgabe erzeugt wird – entweder was gedruckt wird, oder ob ein Laufzeitfehler auftritt (schreiben Sie "Exception"), oder ob der Compiler einen Fehler feststellt (schreiben Sie "Compile-Fehler"). Falls ein gedruckter String Leerzeichen enthält, dann ist die genaue Anzahl/Weite der Leerzeichen unwichtig.

1. `Object tier = new Odobenidae();`
 `((Pinnipedia) tier).praesent();`

2. `Caniformia cx = new Canidae();`
 `cx.jagdbar();`

3. `Caniformia cw = new Ursidae();`
 `System.out.println(cw);`

4. `Arctoidea cy = new Ursidae();`
 `System.out.println((Arctoidea) cy);`
 `if (cy instanceof Pinnipedia) {`
 `cy.praesent();`
 `}`

5. `Ursidae uz = new Ursidae();`
 `uz.geschuetzt();`

6. `Pinnipedia pb = new Otariidae();`
 `((Odobenidae) pb).geschuetzt();`

7. `Pinnipedia pc = new Otariidae();`
 `pc.praesent();`

8. `Arctoidea av = new Odobenidae();`
 `av.praesent();`

9. `Arctoidea at = new Odobenidae();`
 `Pinnipedia pu = (Pinnipedia)at;`
 `pu.praesent();`

Aufgabe 6 (7)

Gegeben sei in Abbildung 1 die EBNF-Beschreibung von *expression*. Für *identifier* gelten die Regeln für Bezeichner (identifiers) und Werte (literal values) in Java. Die EBNF Beschreibung von *expression* unterscheidet sich aber sonst von den in Java zulässigen Ausdrücken.

<i>unary_expression</i>	$\Leftarrow$ <i>unary_operator</i> <i>primary_expression</i>
<i>postfix_expression</i>	$\Leftarrow$ <i>primary_expression</i> <i>postfix_expression</i> + + <i>postfix_expression</i> - -
<i>primary_expression</i>	$\Leftarrow$ (<i>special_expression</i>) <i>identifier</i>
<i>unary_operator</i>	$\Leftarrow$ * !
<i>assignment_expression</i>	$\Leftarrow$ <i>expression</i> <i>assignment_operator</i> <i>special_expression</i>
<i>assignment_operator</i>	$\Leftarrow$ = + =
<i>special_expression</i>	$\Leftarrow$ <i>primary_expression</i> <i>special_expression</i> <i>binary_operator</i> <i>expression</i>
<i>binary_operator</i>	$\Leftarrow$ * / + %
<i>expression</i>	$\Leftarrow$ <i>assignment_expression</i> <i>postfix_expression</i> <i>unary_expression</i>

Abbildung 1: EBNF-Beschreibung von *expression*

Geben Sie für jeden folgenden Ausdruck an, ob er nach der EBNF-Beschreibung von *expression* in Abbildung 1 gültig ist. (Tipp: alle Bezeichner und Werte in diesen Ausdrücken (d.h., *identifier* in Abbildung 1) sind korrekt.)

Ausdruck	Gültig	Ungültig	Ausdruck	Gültig	Ungültig
<code>*(a + b)</code>			<code>(b++) += ((a + c))</code>		
<code>z = *(a + b)</code>			<code>a = (*b * *c)</code>		
<code>y += (z / 7)</code>			<code>y++ * 7</code>		
<code>x = (a = b)</code>			<code>k = m = n</code>		

Aufgabe 7 (4)

Bitte geben Sie für die folgenden Java Programmsegmente die schwächste Vorbedingung (weakest precondition) WP an. Bitte geben Sie die Precondition als Java Expression an. Alle Variablen sind vom Typ `int` und es gibt keinen Over/Underflow.

1. WP: { }

```
w = a;  
x = w + b;  
y = x * 2;  
Q: { y > 0 && b > 10 }
```

2. WP: { }

```
p = 3 * q;  
p = p + 1;  
Q: { p > 15 }
```

3. WP: { }

```
if (a == b) {  
    z = a + b;  
} else {  
    z = 2 * a;  
}  
Q: { z > 0 }
```

Wir wünschen Ihnen alles Gute für den Rest der Prüfungssession und das nächste Semester. Ihr "Einführung in die Programmierung"-Team.